

Database Architecture, Product Storage Schema & Frontend Rendering Pipeline

End-to-End Technical Whitepaper & System Integration Manual • Version 3.4 • Generated: August 25, 2026

Core Architecture Summary: RetailMind operates a high-speed, dual-mode persistence architecture: Google Cloud Firestore provides real-time distributed NoSQL synchronization in production, with seamless zero-dependency fallback to an in-memory/JSON-persisted FastAPI database engine (mock_db.py). Product catalogs sync bidirectionally across web (React 19) and mobile (Flutter) clients with millisecond latency.

1. Executive Summary & Dual-Database Architecture

RetailMind AI eliminates vendor lock-in and cold-start network latency through a dynamic dual-layer storage abstraction. The system runs in two deterministic operational modes without changing a single line of client UI code:

Mode Layer	Engine / Mechanism	Behavior & Persistence	Primary Use Case
Production Mode	Google Cloud Firestore (NoSQL)	Global ACID document transactions, real-time snapshot streams, automatic clustering.	Multi-device live syncing & production deployments.
Fallback / Mock Mode	FastAPI in-memory + JSON disk	Pre-seeded Python dictionaries mirrored to data/registered_customers.json.	Zero-config local development, testing, & offline resilience.

Automatic Database Detection & Failover Logic:

- Environment Flag Probing:** Upon boot, config.py inspects USE MOCK_DB and verifies service certificate files.
- Graceful Exception Interception:** If Firebase credentials are placeholders or network sockets time out, the backend auto-falls back to Mock DB and logs [Firestore] Initialization failed. Forcing Mock DB Mode.
- Health Status Broadcast:** The GET /api/status endpoint returns active database telemetry ({ 'mock_db': true/false, 'status': 'online' }) so clients adjust polling vs. streaming dynamically.

2. Complete Database Schema & Product Data Model

Product entities represent the core commercial catalog in RetailMind. Every item is strongly typed in Pydantic models (backend) and mapped to document records in Firestore or JSON dictionaries in memory:

Field Name	Data Type	Constraints & Default	Functional Description
productId	String	PK, Auto-UUID (prod_...)	Unique immutable product identifier.
name	String	Required, Max 120 chars	Commercial product title displayed on UI.
category	String	Required (Electronics, etc.)	Taxonomic grouping used in frontend filters.
price	Float (Float64)	>= 0.00, 2 decimal precision	Base selling price in localized currency (€/\$) .
stock	Integer (Int32)	>= 0, Default: 0	Live stock quantity; triggers low-stock alerts if < 10.
sku	String	Unique Barcode/SKU string	Stock Keeping Unit code for inventory tracking.

supplier	String	Optional, Default: "	Vendor/Supplier distribution partner name.
image	String (URL)	Fallback Unsplash CDN URL	High-resolution product thumbnail representation.
createdAt	Timestamp / Float	Server Timestamp (time.time())	Creation timestamp for sorting & analytics.

Relational Linkages & Cross-Entity Indexing:

- **Orders (OrderProductModel):** Embedded line-item sub-array referencing name, quantity, and frozen transaction price at checkout time to preserve accounting history.
- **Audit Trail (activity_logs):** Every product creation, stock adjustment, or deletion automatically writes an immutable log record with timestamp and executing user ID.
- **Low Stock Notification Queue:** Adjusting stock below threshold (10 units) automatically synthesizes a high-priority entry into the notification table.

3. Backend API Architecture & Lifecycle Operations

The FastAPI backend provides non-blocking async endpoint handlers with automatic Pydantic request validation and CORS middleware:

Method & Endpoint	Input Payload / Query	Response Code	Lifecycle Action
GET /api/products	?category=...&search=...	200 OK	Queries collection, filters in-memory/Firestore, returns Product array.
POST /api/products	ProductModel (JSON)	200 / 201	Generates UUID prod_..., seeds default image if blank, appends audit log.
PUT /api/products/{id}	ProductModel (JSON)	200 / 404	Updates product entity, recalculates inventory mirror, logs change.
PUT /api/inventory/{id}	{ stock: int }	200 / 404	Adjusts stock level; triggers instant Low Stock Alert if stock < 10.
DELETE /api/products/{id}	URL Path Param product_id	200 / 404	Deletes document from Firestore/memory and registers removal in audit trail.

4. React Web Frontend Data Flow & Display Pipeline

The React 19 web dashboard (web/src/pages/Products.jsx) implements Apple-standard vibrancy design with ultra-low latency rendering:

- **Dual Listener Mechanism:** When Firebase credentials are set, the UI attaches Firestore onSnapshot(collection(db, 'products')) for zero-latency reactive updates. When running on Mock API, it uses high-frequency background polling (fetch(`\${API\_BASE\_URL}/api/products`)).
- **Stock Status Badge Calculation:** Stock thresholds determine dynamic status chips in real time:
 - **In Stock (Green):** stock >= 20 (Active indicator: #34C759)
 - **Medium Stock (Amber):** 10 <= stock < 20 (Warning indicator: #FF9500)
 - **Low Stock (Crimson):** stock < 10 (Urgent alert: #FF3B30)
- **Dynamic Currency Transformation:** The formatPrice() utility dynamically formats raw numerical integers/floats into localized currency strings (e.g. ■1,24,999.00 or \$1,499.00) based on store settings.
- **Fluid Search & Category Filters:** Filtering occurs instantaneously in memory across SKU, name, and category tabs with Framer Motion spring micro-animations.

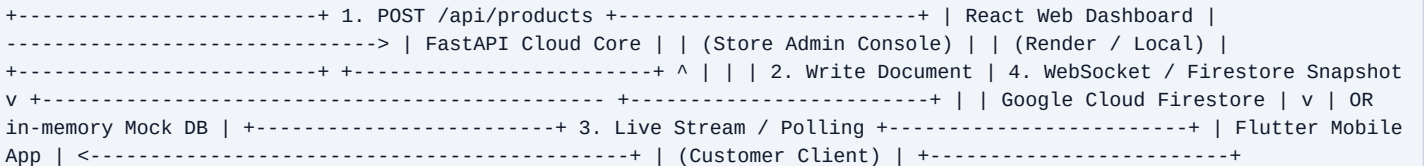
5. Flutter Mobile Client Architecture (Android/iOS/Web)

The Flutter mobile app (mobile/lib/services/firebase_service.dart) handles cross-platform multi-environment discovery:

- **Dynamic Backend Detector (detectBackend()):** Probes available candidate endpoints in sequence (Cloud Hosted Render URL → LAN Wi-Fi IP → Android Emulator IP 10.0.2.2 → Localhost) and locks onto the first healthy responder.
- **Offline Caching with SharedPreferences:** Cart items and custom backend endpoints persist locally across restarts, enabling customer ordering even during intermittent connectivity drops.
- **Customer Sync Engine:** Profile registrations automatically post to /api/customers, linking mobile orders directly to the web dashboard.

6. End-to-End Product Lifecycle & Sync Walkthrough

End-to-End Sequence Walkthrough:



- **Step 1 (Mutation):** Admin creates/modifies a product via React UI → dispatches JSON to POST/PUT /api/products.
- **Step 2 (Persistence & Automation):** Backend validates payload, writes to Firestore/Mock DB, adjusts inventory mirroring, and generates audit log entry.
- **Step 3 (Event Notification):** If stock count drops below 10, backend synthesizes a low-stock alert in the notification queue.
- **Step 4 (Multi-Client Push):** Firestore snapshot listeners fire across both Web and Mobile apps, updating product lists and stock badges instantly without requiring page reloads.